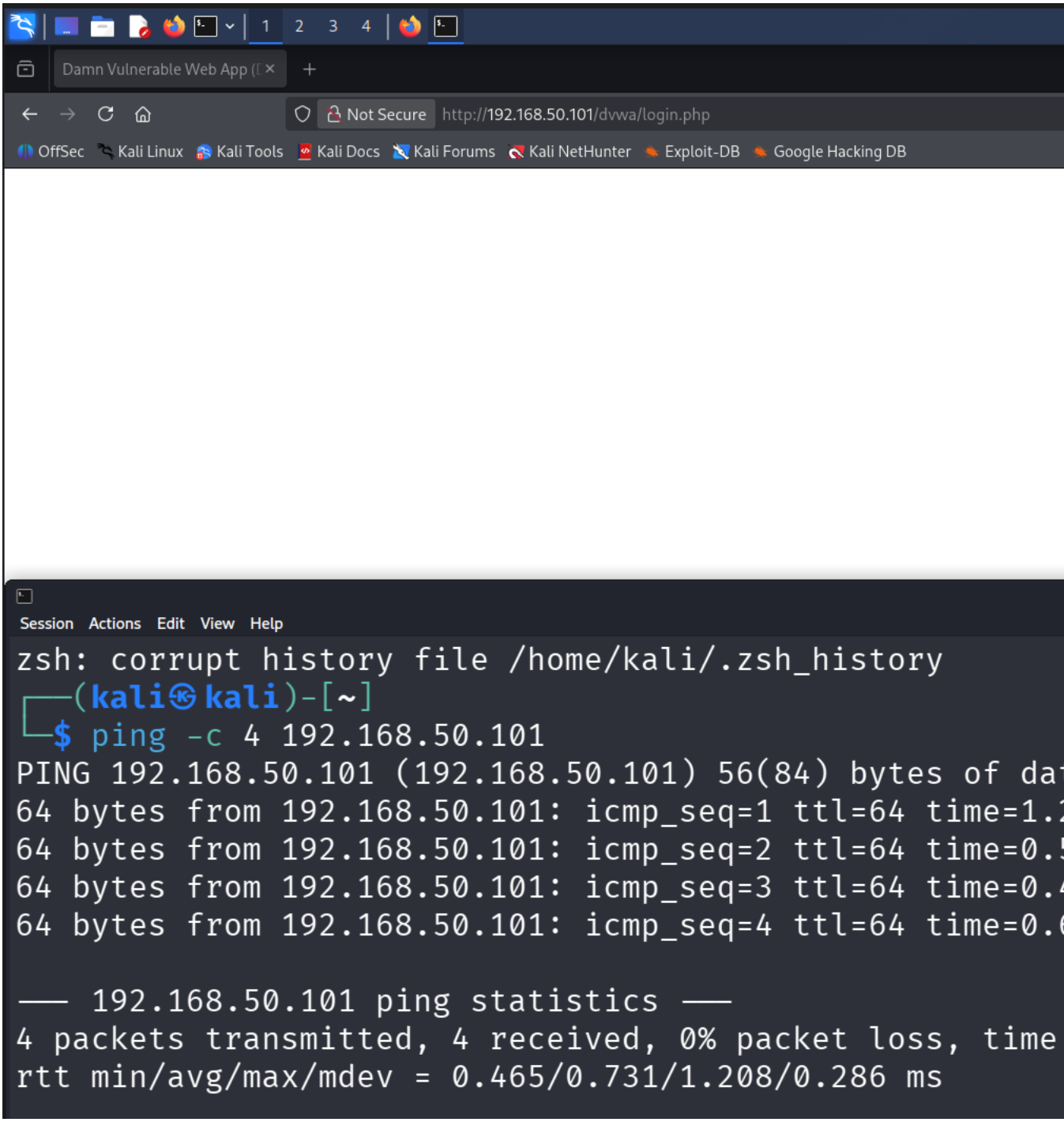
Report: Vulnerabilità su DVWA

1. Configurazione del Laboratorio e Verifica della Connettività

Nella prima fase dell'attività è stata configurata la rete all'interno dell'ambiente virtuale per garantire la comunicazione tra la macchina dell'attaccante e il server ospitante l'applicazione vulnerabile DVWA (IP target: 192.168.50.101).

La connettività di rete è stata verificata tramite il comando ping, come mostrato nella quarta schermata:

- Comando eseguito**:** ping -c 4 192.168.50.101
- Esito**:** 4 pacchetti trasmessi, 4 ricevuti, 0% packet loss



Successivamente, tramite il browser di Kali Linux, è stato effettuato l'accesso all'interfaccia web di DVWA e, all'interno della scheda DVWA Security, il livello di sicurezza è stato impostato su «LOW»

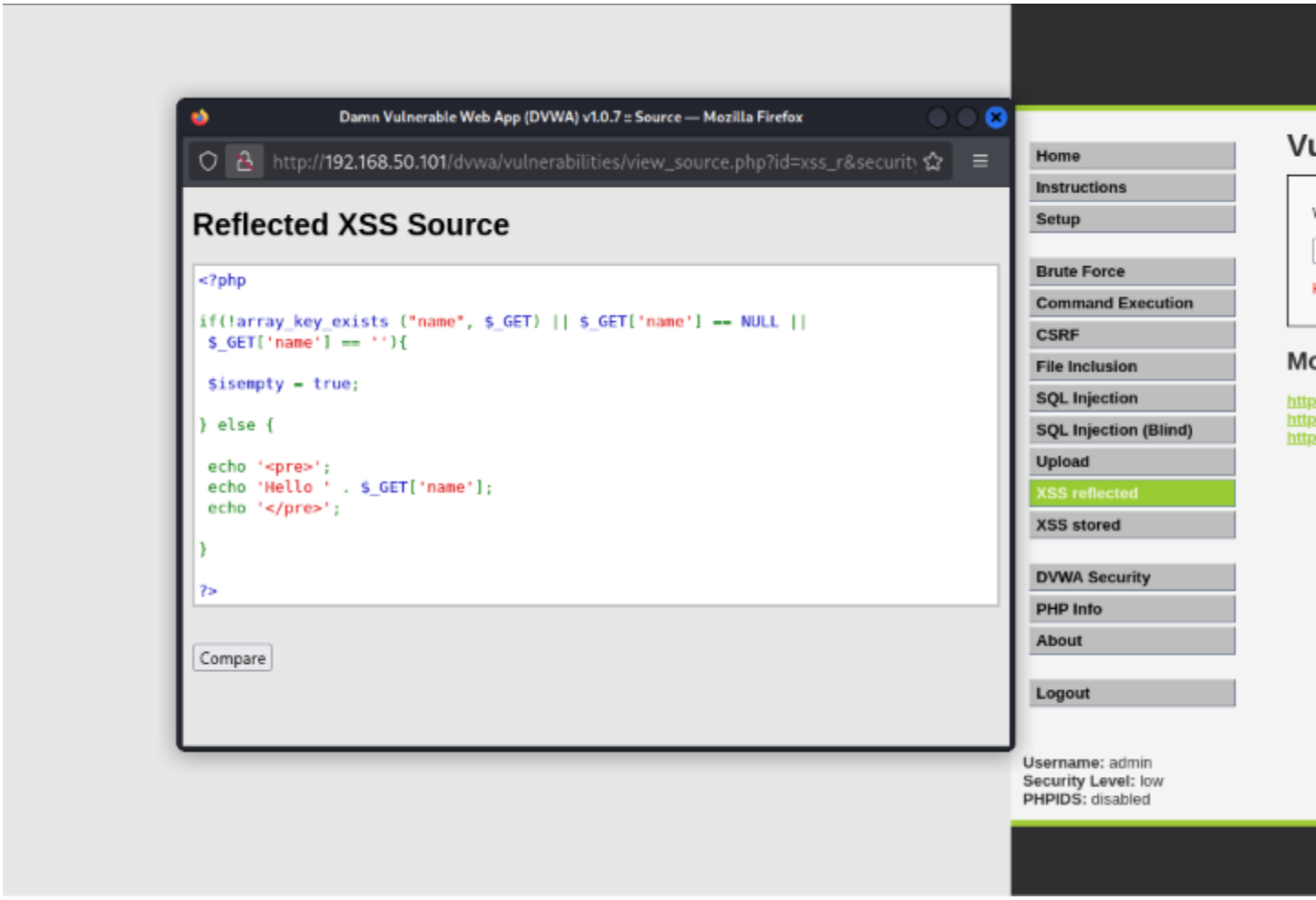
2. Analisi e Sfruttamento delle Vulnerabilità

Le sezioni seguenti illustrano l'approccio teorico e pratico utilizzato per identificare e sfruttare con successo le due vulnerabilità richieste dalla traccia.

Vulnerabilità 1: XSS Reflected (Cross-Site Scripting Riflesso)

Il Reflected XSS si verifica quando un'applicazione web riceve dati all'interno di una richiesta HTTP (ad esempio parametri URL o campi di input) e li include immediatamente all'interno della risposta HTML

Dimensione	Analisi del Laboratorio
Punto di Ingresso	Campo di testo "What's your name?" all'interno della pagina vulnerabilities/xss_r/.
Analisi del Codice Sorgente	Come visibile nella seconda schermata (view_source.php), il frammento PHP lato server esegue un semplice controllo sull'esistenza del parametro $_GET['name']. In caso positivo, esegue direttamente l'istruzione: echo 'Hello ' . $_GET['name']; **Nessuna funzione di filtraggio** (come htmlspecialchars()) viene applicata all'input.
Vettore di Attacco Utilizzato	L'inserimento di una stringa contenente tag HTML/JavaScript malevoli. Ad esempio: <script>alert('XSS')</script> o un payload analogo.
Esito dello Sfruttamento	L'interprete del browser esegue il codice JavaScript iniettato invece di trattarlo come semplice testo, dimostrando la vulnerabilità della pagina. Nel prompt mostrato a schermo, inserendo il nome Luca, l'applicazione restituisce a video Hello Luca, confermando che il testo inserito viene riflesso direttamente nella struttura HTML del blocco <pre>.

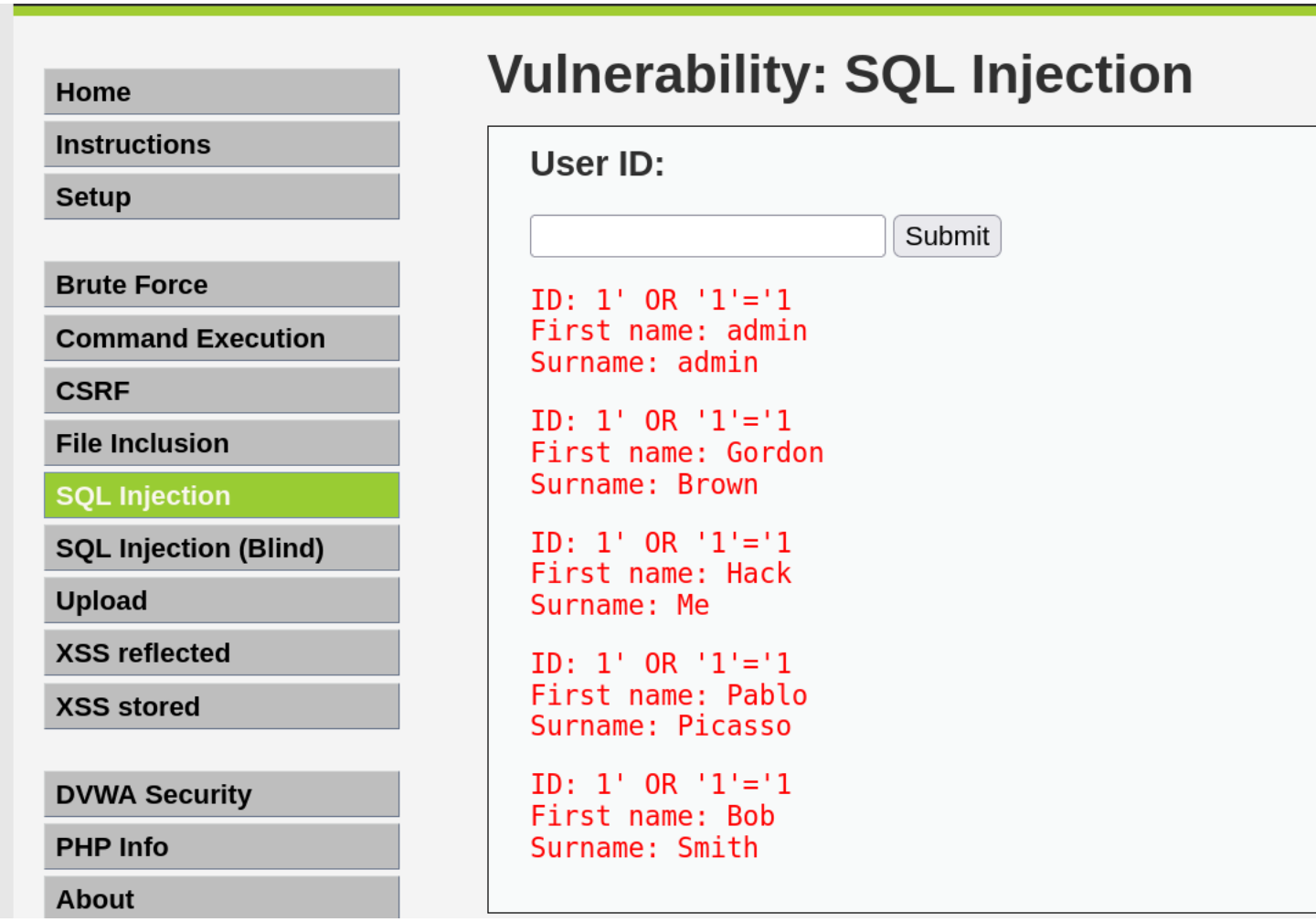


Vulnerabilità 2: SQL Injection (SQLi)

La SQL Injection permette a un attaccante di interferire con le query che un'applicazione esegue verso il proprio database, consentendo di visualizzare dati che normalmente non sarebbero accessibili.

Dimensione	Analisi del Laboratorio
Punto di Ingresso	Campo di testo "User ID:" all'interno della pagina vulnerabilities/sqli/.
Meccanismo di Vulnerabilità	Il backend concatena direttamente l'input dell'utente all'interno della stringa della query SQL (es. SELECT first_name, last_name FROM users WHERE user_id = '$id';) senza adoperare prepared statements o funzioni di escaping.
Payload di Attacco Utilizzato	Per alterare la logica della query SQL è stata inserita una condizione sempre vera preceduta dall'apice di chiusura: 1' OR '1'='1
Logica dell'Attacco	L'inserimento dell'apice interrompe la stringa di controllo originaria. La clausola OR '1'='1 rende l'intera condizione di

	filtraggio della WHERE costantemente vera per ogni singola riga presente nella tabella del database.
Esito dello Sfruttamento	Come documentato nella terza schermata, l'applicazione ha bypassato la restrizione del singolo ID e ha riversato a schermo l'elenco completo di tutti gli utenti registrati nel database con i relativi dettagli: 1. admin admin 2. Gordon Brown 3. Hack Me 4. Pablo Picasso 5. Bob Smith



3. Conclusioni

Il livello di sicurezza impostato su **«LOW»** ha permesso di verificare come l'assenza totale di controlli sugli input degli utenti possa compromettere gravemente sia l'integrità del frontend (tramite XSS) sia la riservatezza e la sicurezza del backend (tramite SQL Injection).

Per mitigare tali rischi in contesti reali è necessario:

1. Adottare giuste predisposizioni per impedire che l'input utente venga interpretato come codice SQL.
2. Implementare un meccanismo rigido di Output Encoding (es. conversione dei caratteri speciali in entità HTML come < e >) prima di stampare a video qualsiasi dato proveniente dall'esterno.